AI Exam Cheat Sheet: how to answer every question type

42172 Introduction to AI · open book · built from your lectures, workshops, key-points sheets, class exercises, Books 1–3 and AIMA. **Find the card** from what the question *gives you* (tap a card name to jump there, tap ↑ index to come back), then follow the steps. Every card has its diagram. ■ yellow = what it means in plain English · green = how to spot it · blue = formula / worked example · red ✗ = traps · purple ★ = shortcuts · grey box = true/false facts the exam likes.

The exam: 100 min · 20% · all modules except Module 1 (Lecture 1 not examined) · questions drawn at random from a bank · types: MCQ, multiple answer, true/false, fill one blank, fill several blanks, matching · bring a log-capable calculator, printed notes, photo ID. **Time:** ~40 questions ≈ 2.5 min each. Do concept questions fast, bank time for calculations. Never leave a blank (no sign of negative marking). **Per type: MCQ** compute first, then look at the options (wrong options = typical slips). **Multiple answer** treat every option as its own true/false. **T/F** watch "always / never / guarantees / any". **Fill blank** give exactly the rounding asked; names spelled as in the slides. **Matching** write both columns on scrap paper, cross off; a right-hand item can be reused.

What Helen flagged for the exam: L10: the inference part is most important, "your exam questions by and large come from it"; the Wumpus world is used in exam questions; **CNF will be tested**. L11: "key focus topic"; the burglary network is used for all questions incl. the exam; "evaluation inside out"; stepwise questions test single steps, so know the whole process. L12: semantic-network reasoning is "the core focus". L10–L12 carry extra weight (no lab). Calculations to practise by hand: search f(n), loss functions, MDP Q/utility, entropy/IG/Gini, probability/Naive Bayes/BN, convolution/pooling/receptive field/parameters, IoU/NMS, RNN update, attention aggregation.

If the question gives you…	Card	If the question gives you…	Card
Graph with edge costs + h values, "expansion order", "f(n)", "path cost returned"	1 Search	Grid with U values, "compute Q(s, Up)", slip 0.1	16 MDP Q
"Is h admissible / consistent?", "which is complete / optimal", BFS/DFS order	2 Heuristics	γ, discounted sum, MEU routes, value / policy iteration	17 Bellman
Hill climbing, SA temperature, GA fitness, roulette, tournament, 8-queens	3 Local	TD update, ADP counts, reward-to-go, Q-learning update, explore/exploit	18 RL
[a+, b−] counts, entropy, Gini, IG, "which attribute"	4 Trees	Word counts per class, |V|, add-one, P(+|text), spam/ham	19 Naive Bayes
w, $\partial L/\partial w$, α; $h(x) = w_1x + w_0$; logistic, cross-entropy	5 GD	x, w_x, h_prev, w_h, tanh value supplied; "× 0.8 for 30 steps"; LSTM gates	20 RNN
Confusion matrix; y vs ŷ table; "which metric"; k-fold, leakage	6 Metrics	One-hot, embedding E, n-gram counts, CBOW, many-to-one	21 Text
Query point + labelled points; SVM w, b, margin; random forest / boosting	6B kNN·SVM	q, keys, values, d_k; or just weights + values	22 Attention
Neuron weights, perceptron update, network a_3 a_4 … ŷ, Δ_5, parameters	7 NN	Raw scores → softmax weight; scale by √d_k	23 Softmax
σ, ReLU, tanh, FVSBN, autoencoder, VAE μ σ KL, GAN, diffusion	8 Act · AE/VAE 8B GAN · diffusion	Experiments count, full joint table, "sum out"	13 Probability
Input grid + kernel, "output[r,c]", max / average pooling	9 Conv	Burglary CPTs, P(B | j, ¬m), enumeration	14 BN
Conv2d(…) → ReLU → MaxPool2d → Flatten → Linear chain	10 Shapes	Causal vs diagnostic, Bayes' rule, independence, query/evidence/hidden	15 Bayes
"How many trainable parameters", receptive field	11 Params	P ⇒ Q, truth table, CNF, resolution, FOL ∀ ∃, UI/EI	24 Logic 24B FOL
Box corners, IoU, NMS + threshold, YOLO, crops; AlexNet/VGG/ResNet	12 Detection	Semantic network, inheritance, ontology, RDF triple, SPARQL	25 KR
Keys to press, σ / tanh / ln lookup values	0 Calculator	Lecture 11 full joint table, marginalising	13 Probability
Not sure which card? → **26 Traps bank** (T/F favourites) and **27 Confusables**	26–27	Last 2 minutes → **28 Before you submit** checklist	28

0 Calculator & numbers to recognise

Book 3 Part I ↑ index

> Most "hard" questions are just these 6 calculator keys used in the right order. Keep 4 decimal places until the end, then round as asked.

Need	Key / rule	Check value
$\log_2 x$	`ln` x ÷ `ln` 2	$\log_2(1/3) = -1.585$
e^x	`SHIFT` + `ln`	$e^{-1} = 0.368$, $e^2 = 7.389$
σ(z)	1 ÷ (1 + e^(−z))	σ(0.6) = 0.646
powers	x^y	$0.8^{30} = 0.00124$
√	√	√2 = 1.414, 1/√2 = 0.707
%	×100 only if asked "as %"	88/5000 = 1.76%

z	−2	−1	−0.4	0	0.5	0.6	1	2
σ(z)	.119	.269	.401	.500	.622	.646	.731	.881

x	0.3	0.5	0.7	0.8	1	1.1
tanh	.291	.462	.604	.664	.762	.800

p	0.1	0.2	0.3	0.5	0.6	0.7	0.8	0.9
ln p	−2.303	−1.609	−1.204	−0.693	−0.511	−0.357	−0.223	−0.105

✗ $\log_2$ for entropy · ln for cross-entropy, KL, GAN loss · $\log_{10}$ (the plain `log` key) is almost never right.

★ max = the biggest VALUE; argmax = WHICH input gives it (policies use argmax). ŷ = prediction, ȳ = average, x_2 = 2nd input (label), x^2 = power.

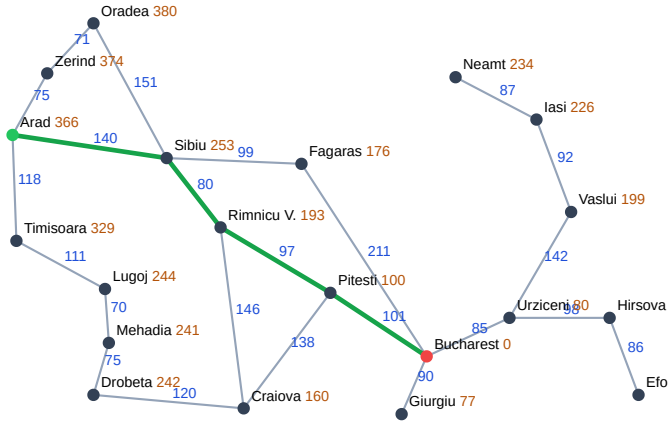
1 Search: UCS · Greedy · A*

Lecture 2 ↑ index

> You walk a map. g(n) = cost already travelled to n. h(n) = a guess of the cost still to go. Each algorithm always expands the frontier node with the smallest score; they differ only in the score.

Spot it: graph, edge costs, h(n) under nodes, "expansion order", "path returned", "f(n) = ___".

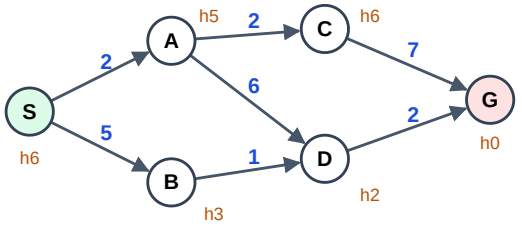
Algorithm	Score (pick smallest)	Frontier
BFS	shallowest (oldest)	FIFO queue
DFS	deepest (newest)	LIFO stack
UCS	f = g(n)	priority queue
Greedy best-first	f = h(n) (ignores g)	priority queue
A*	f = g(n) + h(n)	priority queue



Romania: road costs in blue, h_{SLD} to Bucharest in orange. Green = optimal A* route A–S–RV–P–B = 418.

1. Table: **Step | Expand (g, h, f) | Frontier after | Explored**.
2. Expand the smallest score. Add each neighbour with g_child = g_parent + edge cost and its score.
3. Neighbour already on the frontier with a bigger g? Replace it with the cheaper one. Never re-add explored nodes (graph search).
4. Tie → alphabetical (unless told otherwise). **UCS / A*: stop when G is selected (popped), not when it is added.** BFS: goal test when generated.
5. Path cost = add every edge on the returned path.

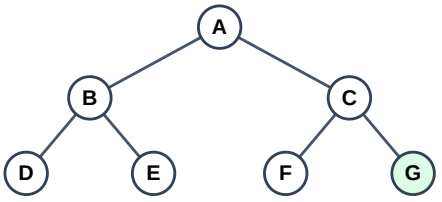
Romania A*: Arad 0+366 = 366 → Sibiu 140+253 = **393**, Timișoara 118+329 = 447, Zerind 75+374 = 449 → expand Sibiu → Rimnicu V. 220+193 = **413**, Făgăraș 239+176 = 415, Oradea 291+380 = 671 → expand RV → Pitești 317+100 = **417**, Craiova 366+160 = 526 → expand Făgăraș (415) → Bucharest 450 → expand Pitești (417) → Bucharest 418 replaces 450 → pop Bucharest **418**: A–S–RV–P–B. **Greedy** (h only): Arad → Sibiu 253 → Făgăraș 176 → Bucharest 0 = path cost 140+99+211 = **450** (not optimal). **UCS** order: A 0, Z 75, T 118, S 140, O 146, RV 220, L 229, F 239, M 299, P 317, C 366, D 374, B 418. **BFS** returns A–S–F–B (3 actions, 450).



Book 2 graph: UCS/A* → S–B–D–G (8); BFS → S–A–C–G (11).

Small graph (S–A 2, S–B 5, A–C 2, A–D 6, B–D 1, C–G 7, D–G 2; h: S6 A5 B3 C6 D2 G0): UCS expands S0, A2, C4, B5, D6 (D 8→6 replaced), G8 → S–B–D–G = 8. A*: f(S)=6, A 7, B 8, D 8, G 8. BFS (goal at generation): S–A–C–G = 11.

✗ Bucharest added at 450 is NOT the answer: 417 was still on the frontier. ✗ Greedy picks g=400,h=50 over g=100,h=80. ✗ "Expansion order" ≠ "path returned". ✗ Don't add h along the path; only the last node's h counts.



Goal G. BFS (FIFO): A B C D E F G · DFS (LIFO, left first): A B D (H I) E (J K) C F (L M) G

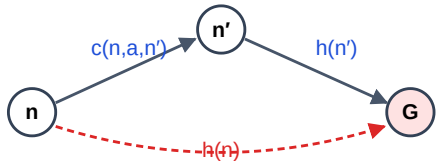
Search problem = 6 parts: state space, initial state, goal test, actions(s), transition model RESULT(s,a), action cost c(s,a,s'). Assumes fully observable, deterministic, known → solution is a fixed action sequence. Romania: 20 cities, 20×20 0/1 adjacency matrix. Tree search may revisit states; graph search keeps an explored set.

2 Admissible · consistent · properties

Lecture 2 ↑ index

Admissible = an honest optimist: the guess is never MORE than the real remaining cost. Consistent = the guess never drops by more than the step you just paid (a triangle rule on every edge).

Admissible: $h(n) \le h^*(n)$ · Consistent: $h(n) \le c(n,n') + h(n')$ for every edge



Consistency = triangle rule: h(n) ≤ c(n,a,n′) + h(n′) on every edge.

1. Admissible: compute the true cheapest cost h* from each node to G (run UCS backwards); compare.
2. Consistent: check every edge in BOTH directions (edges are two-way). One failure → not consistent.

h(S)=16, edge S–B cost 6, h(B)=9: 16 > 6+9 = 15 → **not consistent**. Book 2: h(A)=7, h(D)=0, A–D cost 6: 7 > 6 → admissible (h*(A)=8) but not consistent. R→P: h(R) = 193 ≤ 97 + h(P) = 197 ✓. h(R) = 250 > true cost 198 → not admissible.

	Complete?	Cost-optimal?	Time / Space
BFS	Yes (finite b)	only if all step costs equal	O(b^d) / O(b^d)
DFS	No (tree); yes finite graph	No	O(b^m) / O(bm)
UCS	Yes (costs ≥ ε > 0)	Yes	O(b^(1+⌊C*/ε⌋))
Greedy	Not always	No	O(b^m) worst
A*	Yes	Yes (admissible tree / consistent graph)	exponential worst

★ Consistent ⇒ admissible (never the reverse). A* with h = 0 is UCS. Straight-line distance is admissible (a straight line is never longer than a road). Bigger admissible h (dominates) = fewer nodes expanded. max(h_1, h_2) of admissible heuristics is admissible. b = branching factor, d = depth of shallowest goal, m = max depth.

✗ "Greedy is optimal with an admissible h" = FALSE (450 vs 418). ✗ "A* is optimal with ANY heuristic" = FALSE (an inconsistent h in graph search can make A* return 16 when UCS finds 15). ✗ "BFS returns the cheapest path" = FALSE (fewest steps).

3 Local search · simulated annealing · genetic algorithm

Lecture 3 · Week 3 ↑ index

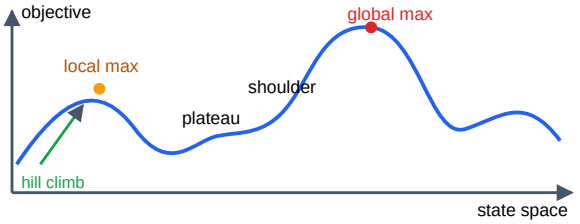
Here the PATH doesn't matter, only the best final STATE (a schedule, a route, a queen layout). Start somewhere and keep improving by small changes.

Spot it: optimisation, objective/cost, neighbour, temperature, population, fitness, 8-queens tuple.

Formulate: decision variables $x_1 \dots x_n$ (one set = one state) · cost Q(x) or fitness R(x) · objective min cost / max fitness · successor (neighbour) function · constraints $low_i \le x_i \le high_i$. Delivery: state = route, neighbour = swap two stops, objective = min distance, constraint = visit each once and return. 20 stops → 20! ≈ 2.4×10^{18} routes. Continuous: min $y = x^2 - 4x + 7$ on [−2, 6] → $(x-2)^2 + 3$ → x = 2, y = 3.

8-queens state $(x_1 \dots x_8)$ = row of the queen in column i · cost = # attacking pairs · pair (i, j) attacks if $x_i = x_j$ or $|x_i - x_j| = j - i$

cost(2,4,7,4,8,5,5,2) = 4 (rows: cols 1&8, 2&4, 6&7; diagonal: cols 3&8) · cost(2,4,6,8,3,1,7,5) = 0 · all in one row or one diagonal = 28 = C(8,2) · cost(1,3,5,7,2,4,6,8) = 1. Each state has 8×7 = **56** neighbours; **92** solutions, **12** fundamental. Hill climbing maximises f = −cost.

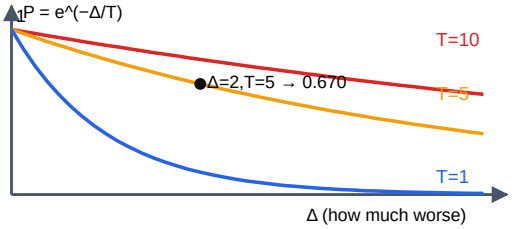


State-space landscape: hill climbing stops at any peak or flat area.

Hill climbing (steepest ascent): move to the BEST neighbour; stop when no neighbour is better. Stuck at local maxima, ridges, plateaus/shoulders → random restarts (also stochastic and first-choice variants; sideways moves cross plateaus).

$y = x^2 - 4x + 7$ from x = −1 (minimise): 12 → 7 → 4 → 3 at x = 2; neighbours 4, 4 → stop. 8-queens 24748552 (cost 4) → best of 56 neighbours 24748551 (cost 2). Values 10 with neighbours 8, 12, 15, 9 → move to 15.

Simulated annealing: better move → always accept; worse by Δ > 0 → accept with $P = e^{-\Delta/T}$



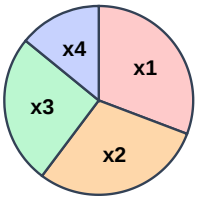
SA: bigger Δ or colder T → smaller acceptance probability.

Δ/T	0.1	0.2	0.5	1	2	3
P	.905	.819	.607	.368	.135	.050

Δ = 10, T = 20 → e^(−0.5) = 0.61; same move at T = 5 → e^(−2) = 0.14 (cooler = pickier). Improves by 5 → P = 1. Slide: E 10 → 12, T = 5: Δ = 2, P = e^(−0.4) = 0.670; random r = 0.52 < 0.670 → move; r = 0.81 → stay.

High T explores (random walk); T → 0 = hill climbing. Cooling too fast → trapped like hill climbing; too slow → many iterations; sensitive to start T and schedule; runs differ; escapes local optima but NO guarantee of the global optimum in one run. Named after annealing metal; minimises "energy".

Roulette: $P_i = f_i / \sum_j f_j$ · **Tournament k:** pick k at random, fittest wins



x1 24/78 = .31 · x2 .29 · x3 .26 · x4 .14

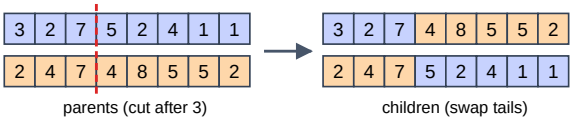
sorted cumulative: x4 [0,11) x3 [11,31) x2 [31,54) x1 [54,78]

randoms 15, 42, 51, 69

→ **x3, x2, x2, x1**

Roulette: slice size ∝ fitness.

f = 24, 23, 20, 11 (total 78) → P = 0.31, 0.29, 0.26, 0.14. Sorted cumulative: x4 [0,11) x3 [11,31) x2 [31,54) x1 [54,78] → randoms 15, 42, 51, 69 pick **x3, x2, x2, x1** (same with 0.14/0.40/0.69/1.0 and 0.164, 0.416, 0.607, 0.894). Tournament k=3 on A70 B55 C40 D30 E20 F10: {A,C,E}→A, {B,D,F}→B, {C,D,E}→C.



One-point crossover; then mutation changes one digit with small probability.

Local beam search: keep the k best states, sharing information among them (stochastic beam: pick ∝ value). Evolutionary algorithms = stochastic beam + natural selection.
GA (Holland 1975): individual = chromosome (bit string) · selection (fitter more likely) → crossover (combine two parents: 110|01 +

001|11 → 110|11, 001|01) → mutation (flip a bit, keeps diversity) → elitism (copy the best). Factors: population size, representation, mixing number (parents, usually 2), selection, recombination, mutation rate, makeup of next generation. Bigger k = more selection pressure, less diversity. Tournament needs no normalising; fitness 0 can't be picked by roulette.

✗ Use the cumulative order the question gives. ✗ "SA guarantees the global optimum" = FALSE. ✗ Local search does NOT keep the path.

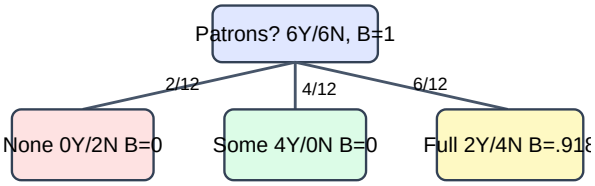
4 Decision tree split: entropy · IG · Gini

Lecture 4 · step-by-step sheet ↑ index

A good question (attribute) splits the examples into PURE groups. Entropy and Gini measure "mixed-up-ness": 0 = pure, max when 50/50. Information gain = how much mixed-up-ness the split removes.

Spot it: [6+, 6−] at the parent, branches with counts, "IG", "Gini", "which attribute is chosen".

$$H = -\sum_k p_k \log_2 p_k \quad \text{Gini} = 1 - \sum_k p_k^2$$
$$IG = H(\text{parent}) - \sum_v \frac{|S_v|}{|S|} H(S_v) \quad \text{Gini reduction} = \text{Gini(parent)} - \Sigma(|S_v|/|S|)\cdot\text{Gini}(S_v)$$

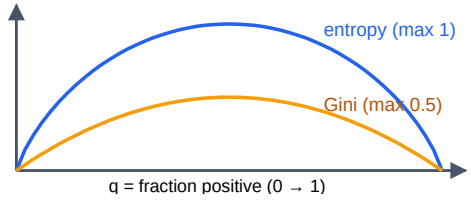


Remainder = 6/12 × 0.918 = 0.459 → Gain(Patrons) = 0.541; Gini reduction 0.278.

1. Parent: H and Gini.
2. Each branch: H and Gini (pure → 0; 50/50 → H = 1, Gini = 0.5).
3. Weight each by branch size ÷ parent size (**not** a plain average) and add.
4. Subtract from the parent. Pick the highest IG = lowest weighted Gini.

Patrons vs Type (12 examples, 6+/6−: H 1, Gini 0.5)
Patrons: None [0+,2−] H0 G0 · Some [4+,0−] H0 G0 · Full [2+,4−] H 0.9183 G 0.4444 → weighted H = 6/12 × 0.9183 = 0.4591 → **IG 0.5409**; weighted Gini 0.2222 → **reduction 0.2778**. Type: French 1/1, Italian 1/1, Thai 2/2, Burger 2/2 → every branch H 1, G 0.5 → IG 0, reduction 0. **Choose Patrons.**
Second level (Full 2Y/4N): Hungry gives Yes 2Y/2N, No 0Y/2N → .918 − 4/6(1) = **.252**. CART binary split Some vs None∪Full: Gini .5 − 8/12(1 − .0625 − .5625) = **.25**.
Play tennis (9+/5−): H = 0.940, Gini 0.459. IG(Outlook) 0.247 (Sunny 2/3, Overcast 4/0, Rain 3/2) · Humidity 0.152 · Wind 0.048 → Outlook.

split	H	Gini	split	H	Gini	split	H	Gini
1:1	1.000	.500	1:4	.722	.320	3:4	.985	.490
1:2	.918	.444	2:3	.971	.480	3:8	.845	.397
1:3	.811	.375	2:5	.863	.408	5:9	.940	.459
1:5	.650	.278	2:7	.764	.346	pure	0	0



Both are 0 for a pure node and largest at 50/50.

★ Same ratio in every child as in the parent → IG = 0. 3 classes (7,4,4): Gini = 1 − (49+16+16)/225 = 0.64; H = 1.530 ($\log_2 3 = 1.585$ is the max for 3 classes, 2 bits for 4).

ID3 = information gain (multiway) · C4.5 = gain ratio + pruning (fixes IG's bias to many-valued attributes) · CART = Gini (classification) / MSE (regression), binary splits, used by scikit-learn. Leaf = class (or average for regression). Deep trees overfit → max_depth, min_samples_leaf, pruning (ccp_alpha). Every root→leaf path is an if-then rule.

✗ IG can never be negative. ✗ A pure branch has entropy 0, not 1. ✗ Plain average of child entropies (unless sizes are equal).

5 Gradient descent · linear & logistic regression

Lecture 4 ↑ index

Training = finding the bottom of a loss "bowl". The gradient says which way is uphill, so step the opposite way. α (learning rate) = step size.

$w \leftarrow w - \alpha \frac{\partial L}{\partial w}$ · linear model $h(x) = w_1 x + w_0$, loss $L = \sum_j (y_j - h(x_j))^2$
$\frac{\partial L}{\partial w_0} = -2\sum(y-h) \quad \frac{\partial L}{\partial w_1} = -2\sum(y-h)\,x$

w = −0.3, ∂L/∂w = 2.3, α = 0.05 → −0.3 − 0.115 = **−0.415**.
Data (1,3),(2,5),(3,7), w_0=0, w_1=1: predictions 1,2,3 → errors 2,3,4 → L = 4+9+16 = 29; $\partial/\partial w_0 = -18$, $\partial/\partial w_1 = -2(2+6+12) = -40$; α = 0.05 → w_0 = 0.9, w_1 = 3.0 (loss falls to 12.83).
One-example form (factor 2 inside α): $w_0 \leftarrow w_0 + \alpha\cdot$err, $w_1 \leftarrow w_1 + \alpha\cdot$err·x. w_0=−1.5, w_1=1.5, α=0.05, (x,y)=(5,1): h = 6, err = −5 → w_0 = −1.75, w_1 = 0.25.

Logistic regression: $h = \sigma(\mathbf{w}\cdot\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}\cdot\mathbf{x}}}$ · loss (binary cross-entropy) $L = -[\,y \ln h + (1-y)\ln(1-h)\,]$ · gradient $(h-y)\,x_i$

w = (0.5, −1, 2) (first = bias, x_0 = 1), x = (1, 1, 0.5), y = 1 → z = 0.5 − 1 + 1 = 0.5 → h = 0.622 → L = −ln 0.622 = 0.474 → bias gradient (0.622 − 1)(1) = −0.378. y = 0, h = 0.8 → L = −ln 0.2 = 1.609.

w_0 = 1, w_1 = 2, example (3, 8), α = 0.1: h = 7, error 1 → w_0 = 1.1, w_1 = 2 + 0.1(1)(3) = 2.3. Logistic z = 1 → p = 0.731; y = 1: loss −ln 0.731 = 0.313; w_1 ← 2 − 0.1(0.731 − 1)(1) = 2.027.

✗ Gradient positive → w goes DOWN. ✗ Forgot α. ✗ Recomputed h with the new w_0 before updating w_1 (compute the error ONCE). ✗ Logistic uses ln, not $\log_2$.

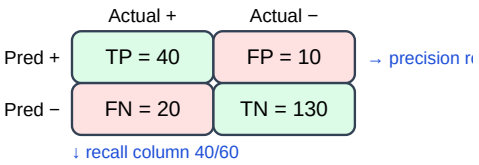
Squared loss is convex (one minimum) → GD works. Too big α overshoots/diverges; too small is slow. SGD / Adam use mini-batches. Threshold 0.5 turns a probability into a class; decision boundary w·x = 0. Linear: MAE/RMSE/R^2; logistic: accuracy, precision, recall, F1, AUC. sklearn C = inverse regularisation strength.

6 Evaluation metrics · workflow · distance

Lecture 4 ↑ index

Classification: count right/wrong in a 2×2 table. Regression: measure the size of errors y − ŷ. Pick the metric that matches the costly mistake.

	Actual +	**Actual −**
Predicted +	TP (hit)	FP (false alarm)
Predicted −	FN (miss)	TN

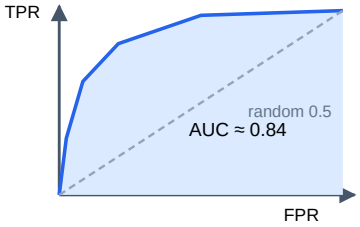


Metric	Formula	Read as
Accuracy	(TP+TN)/all	fraction right (bad if imbalanced)
Precision	TP/(TP+FP)	of predicted +, how many are + (predicted row)
Recall = TPR = sensitivity	TP/(TP+FN)	of actual +, how many found (actual + column)
Specificity = TNR	TN/(TN+FP)	of actual −, how many rejected
FPR	FP/(FP+TN) = 1 − spec	false-alarm rate

F1	2PR/(P+R)	harmonic mean (low if either is low)
Balanced accuracy	(TPR+TNR)/2	fair under imbalance

TP 40, FP 10, FN 20, TN 130: acc 0.85 · P 0.80 · R 0.667 · F1 0.727 · spec 0.929 · FPR 0.071 · BA 0.798.
TP 38, FP 17, FN 18, TN 29: TNR = 29/46 = 0.630, TPR = 38/56 = 0.679 → BA 0.655.

★ Costly false negative (missing cancer) → **recall**. Costly false positive (deleting real email) → **precision**. Balanced classes → accuracy. Both / imbalanced → F1. 99% negatives: "always negative" gets 99% accuracy but recall 0.



ROC: each point = one threshold (0.9 → (.01,.10) … 0.1 → (.60,.95)).

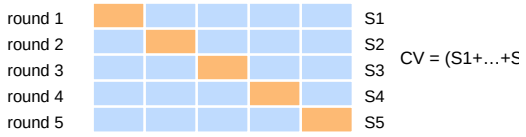
ROC = TPR (y) vs FPR (x) over all thresholds. **AUC**: 1 perfect, 0.5 random, < 0.5 worse than random (inverted); threshold-independent; = P(random positive ranked above random negative). Slide thresholds: 0.9 → (FPR .01, TPR .10), 0.5 → (.10, .60), 0.1 → (.60, .95).

MAE = $\frac{1}{N}\sum|y-\hat{y}|$ · RMSE = $\sqrt{\frac{1}{N}\sum(y-\hat{y})^2}$ · MAPE = $\frac{100}{N}\sum\left|\frac{y-\hat{y}}{y}\right|$ % · $R^2 = 1-\frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2}$

y = (3,5,7,9), ŷ = (2.5,5.5,7,10): errors 0.5, −0.5, 0, −1 → MAE 0.5 · RMSE √0.375 = 0.612 · ȳ = 6, SStot = 20, SSres = 1.5 → R² = 0.925 · MAPE 9.4%. Price 500,000 vs 480,000 → MAPE 4%.
y = (3,5,2,7), ŷ = (2.5,5,4,8): errors .5, 0, −2, −1 → MAE 0.875 · RMSE √(5.25/4) = 1.146 · MAPE 32.7% · ȳ = 4.25 → R² = 1 − 5.25/14.75 = 0.644.

✗ RMSE options: answer without √ = MSE; forgot ÷N; = MAE. ✗ R² CAN be negative (worse than predicting the mean). ✗ MAPE breaks when some y ≈ 0. ✗ AUC depends on threshold = FALSE.

Manhattan $\sum|x_i-y_i|$ · Euclidean $\sqrt{\sum(x_i-y_i)^2}$ · Minkowski $(\sum|x_i-y_i|^p)^{1/p}$ (p=1 Manhattan, p=2 Euclidean)



5-fold CV on the TRAINING set only (orange = validation fold).

Workflow (10 steps): define problem/collect → explore/prepare (encode, scale) → train/test split (e.g. 80/20, test set aside) → choose models + hyperparameter space → tune with **k-fold CV on the training set only** (score = mean of k folds) → pick best → retrain on full training set → **evaluate ONCE on the test set** → residual analysis → deploy/monitor. Using test data while tuning = **data leakage**. Parameters are learned (weights); hyperparameters are set before training (depth, k, α, C).
Learning types: supervised (input–output pairs), unsupervised (patterns, no labels, e.g. clustering), reinforcement (rewards/punishments). Classification = discrete label (spam, COVID +/−, Iris species); regression = number (price, temperature, AUD/USD). **i.i.d.** = independent and identically distributed.
Ockham's razor = simplest consistent hypothesis. Underfit = too simple (bad on train and test); overfit = memorises noise (good train, bad test).

6B kNN · SVM · ensembles

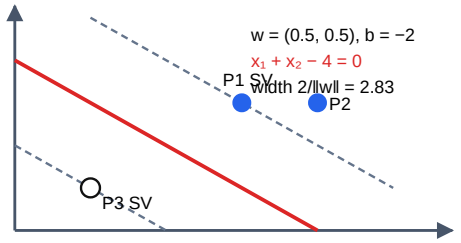
Lecture 4 ↑ index

kNN: copy the majority of the k closest stored examples (no training). SVM: the separating line with the widest "street". Ensembles: many models vote.

kNN query (2,2): A(1,1)+ 1.414, B(3,1)− 1.414, C(2,4)+ 2.000, D(4,4)− 2.828, E(1,3)+ 1.414 → k = 3: A, B, E → **+**. Manhattan to D = 4. Regression kNN = average of the k neighbours' y. House (k = 3, query 90 m², 2 beds): nearest distances 10, 10.05, 30 → prices 220, 260, 180 → **220**. Iris (k = 3) → Setosa.

kNN = instance-based ("lazy"), stores the data, slow prediction, **scale features first**, sensitive to irrelevant features, odd k avoids ties, small k jumpy/overfit, large k smooth/underfit, weights uniform or 1/distance.

SVM: $\min\frac{1}{2}\|\mathbf{w}\|^2$ s.t. $y_i(\mathbf{w}\cdot\mathbf{x}_i+b)\geq 1$ · functional margin $y_i(\mathbf{w}\cdot\mathbf{x}_i+b)$ · geometric = functional / ‖w‖ · street width $2/\|\mathbf{w}\|$ · classify sign(w·x + b)



Max-margin separator: only the support vectors (on the dashed lines) matter.

w = (0.5, 0.5), b = −2, ‖w‖ = 0.7071. P1 (3,3)+ : 1.5+1.5−2 = 1 · P2 (4,3)+ : 1.5 · P3 (1,1)− : −1×(1−2) = 1 → support vectors = P1, P3 (functional margin exactly 1); geometric 1.414, 2.121, 1.414; width 2.828; new (2,3) → 0.5 > 0 → +1. Dual $\alpha_1=\alpha_3=0.25$, $\alpha_2=0$.

Soft margin adds C·Σξᵢ: **large C** = errors expensive → narrower margin, risk overfit; small C = wider margin. **Kernel trick**: replace x·z by K(x,z) to get non-linear boundaries without mapping explicitly (linear, polynomial, RBF with γ = reach; big γ wiggly). Only support vectors decide the boundary. Labels are ±1.

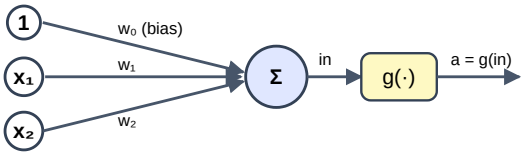
	Bagging / Random Forest	**Boosting (AdaBoost, gradient boosting)**
Built	independently, in parallel, on bootstrap samples (+ random feature subsets per split)	sequentially, each fixes the previous errors (fits residuals)
Reduces	variance	bias
Combine	majority vote / average	weighted sum (learning rate shrinks each tree)

★ Votes Y,N,Y,Y,N → Yes. Regressors 200, 220, 210 → 210. Boosting y = 10: 7 → +2.5 → 9.5 → +0.4 → 9.9. Bootstrap leaves ~37% out (out-of-bag score). Gradient boosting y = 10, 20, 30: F_0 = 20; residuals −10, 0, 10; lr 0.1 → 19, 20, 21. Too many boosting trees overfit (early stopping).

7 Neurons · perceptron · backprop · parameters

Workshop 5 ↑ index

A neuron = weighted sum of inputs + bias, then an activation. Training a network = forward pass (compute ŷ), then pass the error backwards to get each weight's gradient.



A neuron: weighted sum (bias as $w_0\cdot 1$), then activation.

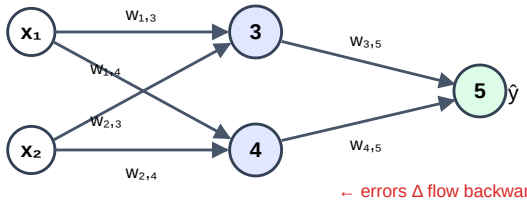
$in_j=\sum_i w_{i,j}a_i+w_{0,j}$ $a_j=g(in_j)$ · Perceptron: $w_i\leftarrow w_i+\alpha(y-\hat{y})\,x_i$ (x_0 = 1 for the bias)

Neuron x = (2, −1), w = (0.5, 1.5), b = 0.25 → z = −0.25 → ReLU 0, σ 0.438, step 0.
Perceptron w = (−0.5, 0.2, 0.4), x = (1,1,1), y = 0, α = 0.1: sum 0.1 ≥

0 → output 1 (wrong) → w = (−0.6, 0.1, 0.3). Correct output → no change. Single perceptron can do AND/OR, **not XOR**.

Sigmoid neuron, squared loss: $\frac{\partial L}{\partial w_i} = -2(y-h)\,h(1-h)\,x_i$

w = (0.1, 0.4, −0.3), x = (1, 2, 1), y = 1, α = 0.5: z = 0.6, h = 0.646, L = 0.126, ∂L/∂w₁ = −0.324 → w₁ = 0.562.



2-2-1 network: forward = weighted sums + activations; backprop sends Δ back along the same links.

Backprop spot it: "inputs 1,2 → sigmoid hidden 3,4 → output 5", a_3, a_4, $w_{3,5}$, $w_{4,5}$, $w_{0,5}$ (bias), target y.

$$\hat{y} = \sigma(w_{3,5}a_3 + w_{4,5}a_4 + w_{0,5}) \cdot \Delta_5 = 2(\hat{y}-y)\,\hat{y}(1-\hat{y}) \cdot \partial L/\partial w_{3,5} = \Delta_5 a_3 \cdot \Delta_3 = \Delta_5\, w_{3,5}\, a_3(1-a_3) \cdot \partial L/\partial w_{1,3} = \Delta_3 x_1$$

a_3 = 0.21, a_4 = 0.24, $w_{3,5}$ = 0.5, $w_{4,5}$ = 1, $w_{0,5}$ = 1, y = 1: in_5 = 1.345 → ŷ = 0.7933 → Δ_5 = 2(−0.2067)(0.7933)(0.2067) = −0.0678 → ∂L/∂$w_{3,5}$ = −0.0142 → Δ_3 = −0.0678×0.5×0.21×0.79 = −0.0056. Then w ← w − α·grad.

Lecture example x = (1, 0), y = 1, α = 0.5: a_3 = 0.622, a_4 = 0.574, ŷ = 0.561, loss 0.193; Δ_5 = −0.2165, Δ_3 = −0.0305, Δ_4 = +0.0212; $w_{3,5}$ 0.6 → 0.667, $w_{1,3}$ 0.5 → 0.515, $w_{2,\cdot}$ unchanged (x_2 = 0).

ReLU hidden, sigmoid out x=(1,2): in_3 = 0.5−0.5+0.1 = 0.1, in_4 = −1+1 = 0 → in_5 = 0.1+0−0.5 = −0.4 → ŷ = 0.401.

★ Sign convention: Book 3 writes Δ = 2(y − ŷ)ŷ(1−ŷ) and updates w ← w + α·a_i·Δ. Same result; just stay consistent.

Dense parameters per layer = $(n_{in}+1) \times n_{out}$ · 4–5–3–2 → 25 + 18 + 8 = 51 · 224×224×3 image → 1000 units = 150,528,000 weights

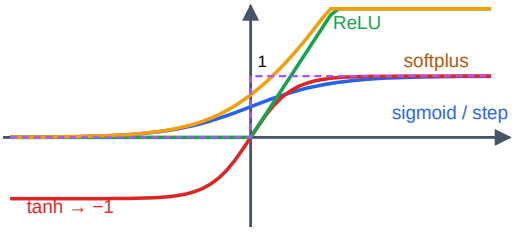
McCulloch & Pitts 1943. Layers: input, hidden, output; bias = weight on a dummy input a_0 = 1. FFN = no cycles; RNN = feedback. **Universal approximation:** two layers (first non-linear, second linear) can approximate any continuous function. **Vanishing gradient:** error signals shrink going back (σ′ ≤ 0.25 per layer; $0.25^{10} \approx 10^{-6}$); ReLU helps. Keras: Sequential → Input → Dense(…, activation) → **compile**(loss, **optimizer='adam'**, metrics) → fit(epochs, batch_size). Hypothesis space = #inputs, #hidden layers, neurons + activation per layer, #outputs. Epoch = one pass over the data; batch = data per gradient step.

✗ Bias added as $w_0 \times 1$. ✗ ŷ is between 0 and 1. ✗ Watch y = 0 vs 1. ✗ Δ_3 uses $a_3(1-a_3)$, not ŷ(1−ŷ).

8 Activations · autoencoders · VAE

Workshop 5 · Lecture 9 ↑ index

Activation	Formula	Range	Note
Step	1 if z ≥ 0 else 0	{0,1}	perceptron; no useful gradient
Sigmoid σ	1/(1+e^−z)	(0,1)	σ(0)=0.5, σ′ = σ(1−σ) ≤ 0.25
tanh	(e^z − e^−z)/(e^z + e^−z)	(−1,1)	zero-centred; tanh(0)=0
ReLU	max(0,z)	[0,∞)	default hidden layer; Leaky = max(0.1z, z)
Softplus	ln(1+e^z)	(0,∞)	"smooth ReLU"; softplus(0) = 0.693



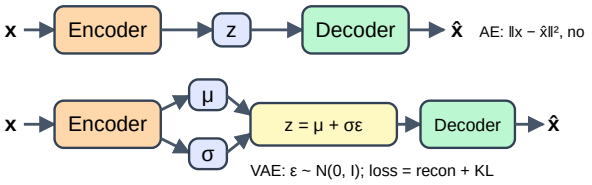
Activations: σ ∈ (0,1), tanh ∈ (−1,1), ReLU ∈ [0,∞), step ∈ {0,1}.

★ Without activations, stacked linear layers collapse into one linear layer. ReLU/pool/flatten have 0 parameters.

> Generative model = learns what the DATA look like, p_model(x) ≈ p_data(x), so it can make new samples. Latent variables z = hidden factors (pose, lighting, identity) that explain an observation x.

Supervised: learn x → y from labels · unsupervised: hidden structure from unlabelled data · generative: model the data distribution to generate. Uses: sample generation, **outlier detection** (train on normal products, flag defects), **debiasing** datasets.

Autoencoder: x → encoder → small z → decoder → x̂; loss ‖x − x̂‖², **no labels**. Smaller z = tighter bottleneck, blurrier output. Denoising AE: noisy in, compare with clean. Uses: compression, denoising, anomaly detection (high reconstruction error). Limitation: not good at generating from random codes.



Autoencoder (top) vs VAE (bottom).

x = (1,0,2), x̂ = (0.5,0,1.5) → loss 0.25+0+0.25 = 0.5.

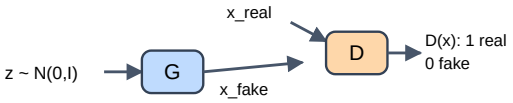
VAE: encoder outputs μ and σ; $z = \mu + \sigma \odot \varepsilon,\ \varepsilon \sim \mathcal{N}(0, I)$ (reparameterisation: lets gradients flow through sampling) Loss = reconstruction + $D_{KL}(q(z|x) \,\|\, \mathcal{N}(0,I))$, $D_{KL} = \frac{1}{2}\sum(\mu^2 + \sigma^2 - 1 - \ln\sigma^2)$

μ = 2, σ = 0.5, ε = −1.2 → z = 1.4 · KL(μ=.5, σ=.8) = ½(.25+.64−1+.446) = 0.168 · AE x = (1,0,.5), x̂ = (.8,.1,.4) → 0.06 · KL(μ=1,σ=1) = 0.5 · KL(0,1) = 0 · KL(0, σ=2) = ½(4 − 1 − ln 4) = 0.807 · KL(μ=0.5,σ=1) = 0.125.

Regularisation gives **continuity** (nearby z → similar output) and **completeness** (any sampled z → meaningful output). β-VAE (β > 1) → disentangled factors. Generate: sample z ~ N(0,I), decode.

8B GAN · autoregressive · diffusion · compare

Lecture 9 ↑ index



GAN: D maximises E ln D(x) + E ln(1 − D(G(z))), G minimises it; equilibrium D = 0.5.

GAN: $\min_G \max_D\ \mathbb{E}[\ln D(x)] + \mathbb{E}[\ln(1 - D(G(z)))]$ — D maximises, G minimises

D(x) = 0.9, D(G(z)) = 0.2: ln 0.9 + ln 0.8 = −0.329 · D(real) = 0.8, D(fake) = 0.3 → ln 0.8 + ln 0.7 = −0.580 · equilibrium D = 0.5 → 2 ln 0.5 = −1.386.

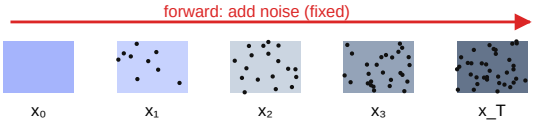
G turns noise into fakes; D outputs P(real). Ideal: $p_G = p_{data}$, D = 0.5 everywhere. **Mode collapse** = G makes one kind of output. After training only G is used. GANs have no explicit density.

Autoregressive: $p(x) = \prod_i p(x_i \mid x_1 \ldots x_{i-1})$ · **FVSBN** \(\hat x_i = \sigma(b_i + \sum_{j<i} W_{ij} x_j)\) · **Diffusion** $x_t = \sqrt{\bar\alpha_t}\, x_0 + \sqrt{1-\bar\alpha_t}\,\varepsilon$, network predicts the noise, loss $\|\hat\varepsilon - \varepsilon\|^2$

May	✓	×	×	×	each row sees only itself
the	✓	✓	×	×	and earlier tokens
Force	✓	✓	✓	×	(lower-triangular mask)
be	✓	✓	✓	✓	

Causal mask: all positions trained in parallel; generation still one token at a time.

Sampling ex: 0.6 × 0.5 × 0.8 = 0.24 · FVSBN $b_4 = -1$, w = (2, −1, .5), x = (1,0,1): σ(1.5) = 0.818 · FVSBN −0.6 + 0.7(1) + 0.2(0) − 0.1(0) = 0.1 → σ = 0.525 · p(May)·p(the|May)·p(Force|May the) = 0.1×0.5×0.4 = 0.02 · $\bar{\alpha} = 0.36$, $x_0 = 2$, ε = 1 → 0.6×2 + 0.8×1 = 2.0 · $\bar{\alpha} = 0.64$, $x_0 = 2$, ε = 0.5 → 1.6 + 0.3 = 1.9.



Diffusion: $x_t = \sqrt{\bar{\alpha}}\, x_0 + \sqrt{1-\bar{\alpha}}\, \varepsilon$; the network learns to predict (and remove) the noise.

Autoregressive: predict → sample → append; **causal mask** → parallel training, sequential generation. GPT = tokens, PixelCNN = pixels, WaveNet = audio, Evo = DNA. Diffusion: add noise forward, learn to denoise; **latent diffusion** (Stable Diffusion) denoises a small latent then decodes once, guided by text. Tokens: Llama 3 = text · AlphaFold 3 = molecular entities · SAM 2 = image patches · RT-2 = observations and actions.

Model	Idea	Strength	Weakness
VAE	encode to a Gaussian, decode	smooth latent, fast	blurry
GAN	G vs D game	sharp	unstable, mode collapse
Autoregressive	next element given the past	exact likelihood, text	slow generation
Diffusion	remove noise step by step	top quality	many steps

9 Convolution output cell · pooling

Lecture 7 · key points ↑ index

A filter (kernel) is a small grid of weights = a "visual question" (is there an edge here?). Slide it over the image; at each place multiply the overlapping numbers and add them up. Big result = pattern found.

Spot it: input grid + kernel, "output[1,0]", "stride", "max / average pooling 2×2 stride 2".

$z[p,q] = \sum_i \sum_j w_{ij}\, x[p+i,\, q+j] + b$ (no flip: technically cross-correlation)

1	1	1	1	1
0	0	0	1	0
0	0	1	0	0
0	1	0	0	0
0	1	0	0	0

step 1: 1−3 = −2

1	1	1	1	1
0	0	0	1	0
0	0	1	0	0
0	1	0	0	0
0	1	0	0	0

step 5: 3−0 = 3 (match)

-1	-1	1
-1	1	-1
1	-1	-1

kernel

-2	-3	1
-2	3	-2
1	-1	-1

output 3×3

Lecture '7' × diagonal kernel, S1 P0: each output = Σ(1s under +1) − Σ(1s under −1).

1. output[r, c] → window's top-left at input[r·S, c·S] (S = stride).
2. Multiply each window cell by the kernel cell on top of it. Skip zeros.
3. Add everything (+ bias if given) = that output cell.

Window [3 1 3 / 1 1 1 / 2 0 2] × kernel [0 −1 −1 / −1 0 1 / 0 −1 1] → (0−1−3) + (−1+0+1) + (0+0+2) = **−2**.
Lecture '7' (rows 11111/00010/00100/01000/01000) × diagonal [−1 −1 1 / −1 1 −1 / 1 −1 −1], S1 P0 → 3×3 map [−2 −3 1 / −2 3 −2 / 1 −1 −1]; out[1,1] = 3 (best match).
X = [1 2 0 1 / 0 1 3 1 / 2 1 0 0 / 1 0 1 2], K = [1 0 / 0 −1] → [0 −1 −1 / −1 1 3 / 2 0 −2] (top-left − bottom-right of each 2×2).

★ Kernel [1, 0, −1] in every row = left column sum − right column sum (vertical edges). [−1 −1 −1 / 0 0 0 / 1 1 1] = horizontal edges. Input only 0/1 → count 1s under +1 minus 1s under −1.

Max pool = biggest in the window · Average pool = sum ÷ (K×K) · **no parameters** · pool size K → stride K by default

230	135	0	0
231	146	0	0
207	136	0	0
163	120	0	0

max →

231	0
207	0

2×2, stride 2:
max(230,135,231,146) = 2
no learned weights;
512×512 → 256×256

Max pooling keeps the strongest activation in each window.

[1 3 2 4 / 5 6 1 2 / 7 2 9 0 / 1 8 3 4], 2×2 stride 2 → max [6 4 / 8 9], average [3.75 2.25 / 4.5 4]. Lecture: rows (230,135,0,0), (231,146,0,0),(207,136,0,0),(163,120,0,0) → max [231 0 / 207 0]. 512×512 → 256×256. Averages of 4 always end .00 .25 .50 .75.

✗ [0,1] = row 0, column 1 = top-RIGHT block. ✗ Stride 2 → window jumps 2 (no overlap for 2×2). ✗ Don't flip the kernel.



Each filter spans ALL input channels → one map; output depth = number of filters; params (27+1)×8 = 224.

Why convolution (not dense): dense layers **flatten** (lose the 2-D neighbourhood) and **parameters explode**; convolution = **locality** + **weight sharing** + **hierarchy** (edges → parts → objects). ReLU after conv (max(0,z)). Deeper layers: larger receptive field, more abstract features.

10 Track the tensor shape

Lecture 7 ↑ index

Spot it: "Input 1×28×28. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → …"

$$N_{out} = \left\lfloor \frac{N+2P-K}{S} \right\rfloor + 1 \quad (\lfloor\ \rfloor = \text{round DOWN})$$

Read it as	K	S if missing	P if missing	Channels
Conv2d(C_in, C_out, K, stride, padding)	3rd	1	0	→ C_out
MaxPool2d(K)	1st	= K	0	unchanged

Layer	Calculation	C×H×W
Input		1×28×28
Conv(1,8,3,S2,P1)	⌊(28+2−3)/2⌋+1 = 14	8×14×14
ReLU	no change	8×14×14
MaxPool(2)	(14−2)/2+1 = 7	8×7×7
Conv(8,32,3,P1)	(7+2−3)/1+1 = 7	32×7×7
MaxPool(2)	⌊(7−2)/2⌋+1 = 3	32×3×3
Flatten	32×3×3	288
Linear(288,10)		10 logits

N	K	P	S	out	N	K	P	S	out
7	3	0	1	5	224	7	3	2	112 (ResNet)
32	5	0	1	28	227	11	0	4	55 (AlexNet)
32	3	1	1	32 (same)	64	5	2	2	32
28	3	1	2	14	13	3	1	2	7



FashionCNN shapes (Conv(1,16,3,S2,P1) → MaxPool2 → Conv(16,32,3,P1) → MaxPool2 → Linear(288,10)).

★ K3 P1 S1 → same size · K3 P1 S2 → N/2 rounded UP (28→14, 7→4) · pool 2 → N/2 rounded DOWN (7→3). C changes only at conv (= number of filters); next conv's C_in = last conv's C_out. Flatten = C×H×W. 64×64×3 with 8 filters 3×3×3 → 62×62×8. FashionCNN params: conv1 (9+1)×16 = 160 · conv2 (144+1)×32 = 4640 · linear 288×10+10 = 2890. Softmax of logits (2, 1, 0.1) → (0.659, 0.242, 0.099); CE = −ln 0.659 = 0.417. Stride-1 version of the classifier: 1×28×28 → 16×28×28 → pool 16×14×14 → 32×14×14 → pool 32×7×7 → flatten 1568.

PyTorch batch shape [batch, channels, height, width]. Model returns **raw logits** (CrossEntropyLoss applies softmax itself). Training step: zero_grad → forward → loss → backward → step. model.train() / model.eval() + torch.no_grad(). Lab habit: print x.shape after each block.

11 Parameter count · receptive field

Lecture 7 · key points ↑ index

Each filter looks through ALL input channels, so a 3×3 filter on RGB has 3×3×3 = 27 weights (+1 bias). The same filter is reused everywhere, so image size never adds parameters.

Conv = $(K_h K_w C_{in} + 1) \times$filters (no bias stated → no +1) · Dense = $(n_{in} + 1) \times n_{out}$

Question	Answer
two 3×3 filters, RGB, bias	(27+1)×2 = **56**
8 filters 3×3×3, bias (64×64×3 → 62×62×8)	28×8 = 224
four 5×5, RGB, bias	(75+1)×4 = 304
16 × 5×5, RGB, bias	76×16 = 1216
32 × 3×3 on 16 channels, bias	145×32 = 4640
32 × 1×1 on 64 channels, bias	65×32 = 2080
eight 3×3, 16 channels, no bias	144×8 = 1152
64 × 3×3, RGB, bias	28×64 = 1792
64 × 3×3 on 32 channels, bias	289×64 = 18,496
64×64×1 flattened → dense 100, no bias	409,600

Chain of Card 10: Conv1 (3·3·1+1)×8 = 80 · Conv2 (3·3·8+1)×32 = 2336 · Linear (288+1)×10 = 2890 · total 5306.

✗ 1728 = forgot bias · 1729 = one bias in total · 640 = forgot the 3 channels · (3×3+1)×2 = 20 = forgot channels. Pooling/ReLU/flatten = 0.

Receptive field, go backwards from 1: $r_{before} = (r_{after} - 1) \cdot S + K$ · stride-1 stack of L layers of 3×3: 1 + 2L

conv3 → pool2 S2 → conv3: 1 → (1−1)·1+3 = 3 → (3−1)·2+2 = 6 → (6−1)·1+3 = **8**. Two 3×3 → 5×5, three → 7×7, four → 9×9.

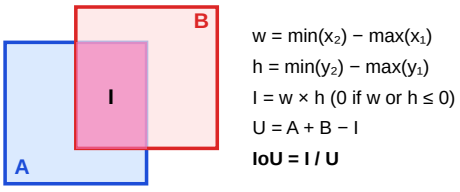
12 IoU · NMS · detection · CNN architectures

Lecture 7 · key points ↑ index

IoU = how much two boxes overlap (0 none, 1 identical). NMS removes duplicate boxes on the same object: keep the most confident, delete its same-class near-copies.

overlap w = min(x_2) − max(x_1), h = min(y_2) − max(y_1) (≤ 0 → no overlap) · $IoU = \frac{I}{A+B-I}$

(0,0,4,4) vs (1,0,5,4): 12/20 = 0.6 · identical boxes → 1 · disjoint → 0 · A (0,0)–(6,4), B (2,0)–(8,4): areas 24, I = 4×4 = 16, U = 32 → **0.50**; rule "suppress if > 0.40" → suppress B.
A (1,2)–(7,6), B (4,3)–(9,8): w 3, h 3, I 9, A 24, B 25, U 40 → 0.225 · (1,1)–(5,5) vs (3,2)–(7,6): 6/26 = 0.231 · (0,0)–(4,4) vs (2,2)–(6,6): 4/28 = 0.143 · (0,0)–(10,10) vs (5,0)–(15,10): 50/150 = 0.333.



A (0,0)–(6,4), B (2,0)–(8,4): I = 16, U = 32 → IoU 0.50.

1. Sort boxes by score; keep the top one.
2. Delete every lower box of the SAME CLASS with IoU(top, box) > threshold (≥ if the rule says ≥).
3. Take the next highest remaining box; keep; repeat.

P .92, Q .85, R .70; IoU(P,Q) = .40, IoU(P,R) = .55. Rule "> 0.40" → keep P, Q, drop R. Rule "≥ 0.40" → only P. Cars A .90, B .80, C .75 + person D .70: IoU(A,B) = 9/23 = .39 keep, IoU(A,C) = .75 drop, D never compared → A, B, D.

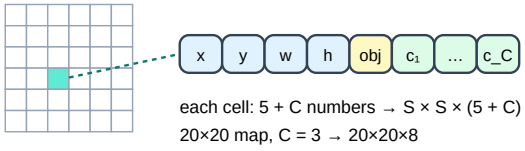
✗ Union = A + B − I (not A + B). ✗ Box inside box: IoU = small/big. ✗ Centre format (x, y, w, h) → corners x ± w/2 first. ✗ IoU exactly =

threshold with ">" → NOT suppressed. ✗ NMS across classes = FALSE.

Sliding-window crops = locations × scales × aspect ratios · YOLO output = S × S × (5 + C)

80×80 × 3 × 3 = 57,600 crops · 40×40 × 3 × 4 = 19,200 · YOLO 20×20 map (400 locations) with C = 3 → 20×20×8; C = 80 → 400 × 85 = 34,000 numbers (box 4 + objectness 1 + C class scores).

Task	Output
Classification	one label for the whole image ("what")
Object detection	variable number of (class, box, confidence)
Semantic segmentation	a class for every pixel; logits [N,C,H,W] → argmax(dim=1) → [N,H,W]; encoder down, decoder up

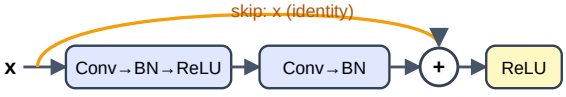


each cell: 5 + C numbers → S × S × (5 + C)
20×20 map, C = 3 → 20×20×8

YOLO: run the CNN once; every feature-map cell predicts a box + objectness + class scores.

YOLO: run the CNN **once**, predict everywhere on a shared feature map. Threshold too low → real neighbours suppressed; too high → duplicates survive. Objectness = is there any object; class score = which class.

Net	Year	Key idea
AlexNet	2012	large early kernels, ReLU, dropout, GPU training breakthrough
VGG	2014	repeated small 3×3 convs in regular blocks
ResNet	2015	skip connections y = ReLU(F(x) + x): if F = 0 the block passes x; gradients get a shortcut



ResNet block: y = ReLU(F(x) + x); if F = 0 the block passes x on.

Feature hierarchy (ResNet-18): 3×224×224 → early 64×112×112 (edges) → middle 128×28×28 (textures, parts) → deep 512×7×7 (abstract) → global average pooling → 512-vector.
Fine-tuning: load pretrained weights → replace the classifier head → train with a small learning rate (head first) → unfreeze selected layers → evaluate on held-out data. "Pretraining is a starting point, not a guarantee." **Sensors:** RGB camera → image · thermal → thermal image · depth camera → depth map · LiDAR → point cloud · radar → range–Doppler map · event camera → event stream · CT/MRI → slices/volume. Pipeline: signal → sensor → data → model → action. CV's two big problems: **reconstruction** and **recognition**.

13 Probability from tables

Lecture 11 ↑ index

A full joint table lists the probability of every combination. Any question = pick the rows that match and ADD; "given" = divide by the rows of the condition.

Word	Rule	Example
CHANCE	favourable/total	die P(4) = 1/6
NOT	1 − p	rain .3 → .7
OR (exclusive)	add	1 or 6 = 1/3
OR (general)	P(A)+P(B)−P(A∧B)	.3+.5−.15 = .65
AND (independent)	multiply	head and 6 = 1/12
GIVEN	P(A∧B)/P(B)	King | face = 4/12

X	Y	Z	P
x1	y1	z1	.06
x1	y1	z2	.09
x1	y2	z1	.14
x1	y2	z2	.11
x2	y1	z1	.18
x2	y1	z2	.12
x2	y2	z1	.16
x2	y2	z2	.14

P(x1) = .06+.09+.14+.11 = .40
P(y1) = .45 · P(z1) = .54
P(x1|y1) = (.06+.09)/.45 = .333
P(x2, z2) = .12+.14 = .26

Marginalise = add the matching rows; conditional = divide by the condition's rows.

P(event) = count / total (×100 for %) · $P(X{=}x) = \sum_y \sum_z P(x,y,z)$ · $P(A|B) = \frac{P(A,B)}{P(B)}$

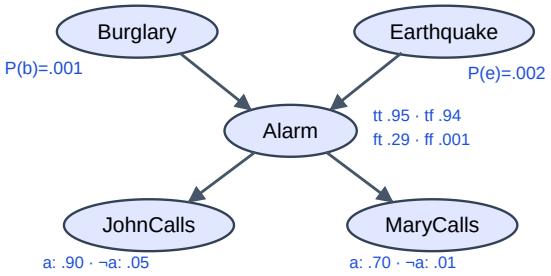
88/5000 = 1.76% · 233/5000 = 4.66% · Lecture table x_1 rows 0.06+0.09+0.14+0.11 = 0.40 · $P(y_1)$ = 0.45 · $P(x_1|y_1)$ = 0.15/0.45 = 0.333 · $P(z_1)$ = 0.54 · 5 binary variables → 2^5 = 32 rows (31 free numbers); 20 variables → 1,048,576.
Burglary experiments: N = 99,995,505,198; row (J,M,A,B,E) = (T,F,T,T,F) occurred 25,329,240 → 0.000253. P(b | j, ¬m) from the table = 0.000256774 / (0.000256774 + 0.049798101) = **0.0051**.

★ "Summing out" = adding. Prior = belief before evidence; posterior = after. The full table does not scale (2^n rows, unnatural to specify, needs huge numbers of experiments) → Bayesian networks.

14 Burglary BN: P(B | evidence) by enumeration

Lecture 11 · class exercises ↑ index

P(B)	P(E)	B E → P(a)	A → P(j)	A → P(m)
.001	.002	tt .95 · tf .94 · ft .29 · ff .001	t .90 · f .05	t .70 · f .01



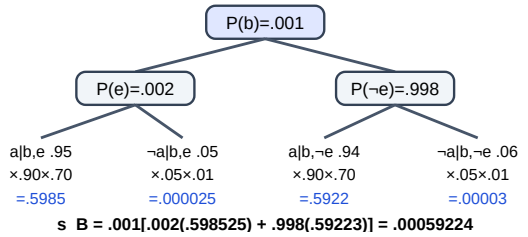
Burglary network: 1 + 1 + 4 + 2 + 2 = 10 numbers (vs 31 for the full table).

Joint: $P(B,E,A,J,M) = P(B)P(E)P(A|B,E)P(J|A)P(M|A)$ · $s_B = P(b)\sum_e P(e)\sum_a P(a|b,e)P(j|a)P(m|a)$ · $P(b|\text{ev}) = \frac{s_B}{s_B + s_{\neg B}}$

1. L(a) = P(j|a)·P(m|a) and L(¬a) = P(j|¬a)·P(m|¬a). For "¬m" use 1 − P(m|·).
2. For each E value: P(a|b,e)·L(a) + (1 − P(a|b,e))·L(¬a).
3. Weight: P(e)·[e row] + P(¬e)·[¬e row]; multiply by P(b) → s_B.
4. Same with B = false (rows .29, .001) and P(¬b) = .999 → s_¬B.
5. Normalise.

P(b | j, m): L(a) = .63, L(¬a) = .0005 · b: e → .95(.63)+.05(.0005) = .598525, ¬e → .94(.63)+.06(.0005) = .59223 → s_B = .001[.002(.598525)+.998(.59223)] = **.00059224** · ¬b: e → .29(.63)+.71(.0005) = .183055, ¬e → .001(.63)+.999(.0005) = .0011295 → s_¬B = **.00149186** → P = **0.2842**.

P(b | j, ¬m): L(a) = .9×.3 = .27, L(¬a) = .05×.99 = .0495 → s_B = 2.568e−4, s_¬B = 4.980e−2 → **0.0051**.



s B = .001[.002(.598525) + .998(.59223)] = .00059224

Enumeration tree for s_B: multiply down a path, add sibling branches, evaluate inside out.

Known answers		Known answers	
P(b | j, m)	0.2842	P(j, m, a, ¬b, ¬e)	0.000628
P(b | j, ¬m)	0.0051	P(¬j, m, a, b, ¬e)	6.57e−5
P(b | j)	0.016	P(j, m, a, b, ¬e)	0.000591
P(e | j, m)	0.176	P(all false)	0.936
P(b | a)	0.374	P(a)	0.002516
P(a | b)	0.940	P(j)	0.05214
P(a | ¬b, e) (CPT row f,t)	0.29	BN numbers vs full table	10 vs 31

★ Query = B · evidence = J, M · hidden (summed out) = E, A. Along a tree path: **multiply**. Branches of a hidden variable (e/¬e, a/¬a): **add**. Evaluate inside out. P(B) moves outside both sums because it doesn't depend on E or A. Then divide each score by s_B + s_¬B (don't just pick the bigger).

✗ Using P(m|a) when m is false (use 1 − 0.70 = 0.30). ✗ Using the B = true CPT rows for the ¬b score. ✗ Forgetting P(¬e) = 0.998. ✗ "Enumeration is approximate" = FALSE (it is exact but repeats work, e.g. P(j|a) four times; variable elimination/clustering reuse results; sampling = approximate, out of scope).

15 Bayes' rule · directions · independence · building a BN

Lecture 11 ↑ index

$P(C|E) = \frac{P(E|C)\,P(C)}{\sum_c P(E|c)P(c)}$ — posterior = likelihood × prior ÷ evidence

Disease prior 1%, test 90% sensitive, 5% false positives: P(+) = .009 + .0495 = .0585 → P(d|+) = **0.154**. Prior 5%, 80%, 10% → 0.04/0.135 = **0.296**. Spam 0.4×0.3 = .12 vs ham 0.6×0.05 = .03 → 0.8. Meningitis 0.7 × (1/50000) / 0.01 = 0.0014.

Direction	Means	Example
Causal	cause → effect, P(effect | cause) — what we usually KNOW	P(S | D), P(alarm | burglary)
Diagnostic	effect → cause, P(cause | effect) — what we WANT	P(D | S), P(burglary | alarm)

Independent: $P(X,Y) = P(X)P(Y)$ · Conditionally independent given Z: $P(X|Y,Z) = P(X|Z)$

BN = **DAG** (no cycles) · node = random variable · arrow parent → child · each node has a CPT P(X | parents) (root: P(X)) · child node = has a CPT given its parents · edge = association between two nodes · arrow = the parent→child link. A node with k Boolean parents needs 2^k rows (3 parents → 8). Burglary: 1+1+4+2+2 = 10 numbers vs 2^5-1 = 31. Given its parents, a node is independent of its other predecessors (J ⊥ B | A). B and E are independent until A is observed.

Building: chain rule $P(x_1 \dots x_n) = \Pi\, P(x_i \mid x_1 \dots x_{i-1})$ → choose variables → order **causes before effects** (most compact) → pick a minimal parent set for each node → draw arrows → fill CPTs. **Inference**: query X, evidence e, hidden Y: P(X|e) = α Σ_y P(X, e, y). Uncertainty: partially observable (drone in fog), non-deterministic (trading agent), both (city driving).

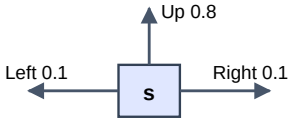
16 MDP grid: compute Q(s, a)

Workshop 6 ↑ index

Q(s, a) = how good it is to TRY action a in state s: average over where you might land of (reward now + discounted value of the landing square).

Spot it: small grid with U values around s, "intended 0.8, slips 0.1 each side", R = −0.04, γ = 1.

$$Q(s,a) = \sum_{s'} P(s'|s,a)\,[\,R(s,a,s') + \gamma\, U(s')\,]$$



try Up: never slips backwards

You try	0.8 goes	0.1 slips	0.1 slips
Up	Up	Left	Right
Down	Down	Left	Right
Left	Left	Up	Down
Right	Right	Up	Down

1. Find the 3 landing cells. Wall or edge → stay in s → use U(s).
2. Each row: P × (R + γU). 3. Add.

above 0.53, left 0.83, s 0.44, right = wall: 0.8(−0.04+0.53) + 0.1(−0.04+0.83) + 0.1(−0.04+0.44) = 0.392 + 0.079 + 0.040 = **0.511**. 4×3 world (1,1): Up = −0.04 + 0.8(0.762) + 0.1(0.705) + 0.1(0.655) = **0.706** · Left 0.671 · Down 0.660 · Right 0.631 → best Up. (3,3) Right = −0.04 + 0.8(1) + 0.1(0.918) + 0.1(0.660) = 0.918.

3	0.812 →	0.868 →	0.918 →	+1
2	0.762 ↑	WALL	0.660 ↑	−1
1	0.705 ↑	0.655 ←	0.611 ←	0.388 ←
	1	2	3	4

4×3 world, γ = 1, R = −0.04 (AIMA/Book): converged U and optimal policy. Start (1,1).

★ Workshop 6 slide version uses U = R + γ·ΣPU with values .852 .908 .958 / .802 wall .700 / .745 .695 .651 .428 (each = textbook value + 0.04, same policy; (3,1) goes Left: Left .651 > Up .633). Use whichever grid the question gives.

★ Same R everywhere → Q = R + γ(0.8U_1 + 0.1U_2 + 0.1U_3). Living reward −0.04 makes the agent prefer short paths to +1.

✗ Slips are 90° sideways, never backwards. ✗ Probabilities must add to 1. ✗ Forgot the wall-bump "stay" term.

17 Bellman · value/policy iteration · discount · MEU

Workshop 6 ↑ index

$$U(s) = \max_a Q(s,a) \cdot \pi^*(s) = \arg\max_a Q(s,a) \cdot U(s) = \max_a \sum_{s'} P(s'|s,a)[R + \gamma U(s')]$$

a: 0.7(2 + 0.8·7) + 0.3(1 + 0.8·(−3)) = 5.32 − 0.42 = 4.90 · b: 0.7(12.8) + 0.2(6.4) + 0.1(12.4) = 11.48 → U = 11.48, π = b. 0.7(−1 + 0.9·10) + 0.3(−1 + 0) = 5.3.

Discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \cdots$ · r forever = $r/(1-\gamma)$

0, 0, 10, γ 0.5 → 2.5 · 1,1,1,10, γ 0.9 → 10.0 · 5,5,5, γ 0.5 → 8.75 · 2 forever, γ 0.8 → 10 · −1 forever, γ 0.9 → −10. γ = 0 → only the immediate reward; γ = 1 → plain sum.

MEU: $EU(a) = \sum P(\text{outcome})\, U(\text{outcome})$, choose the max

Route A 0.8(10) + 0.2(−10) = 6 vs B certain 7 → **B**. X: 0.5(100) + 0.5(−20) = 40 vs Y 0.9(40) = 36 → X. A: 0.6(20) + 0.4(−5) = 10 vs 8 → A.

Discounted −.04, −.04, +1 with γ = .9 → −.04 − .036 + .81 = 0.734. VI sweep 1 at (3,3) Right: .8(1) + .1(−.04) + .1(−.04) = .792 (from U = −0.04 start); sweep 2 at (2,3) Right: .8(−.04 + .792) + .2(−.08) = .586.

Value iteration: start U = 0 (terminals ±1); each round U_new(s) = max_a Q using only OLD U; repeat until change δ < ε(1−γ)/γ. (3,3) Right: 0.76 → 0.832 → 0.890 → … 0.918. With γ = 0.9 first update = 0.68. Equations are non-linear (max). **Policy iteration:** evaluate fixed π (no max → linear equations) → improve with argmax → stop when π doesn't change; often converges faster.

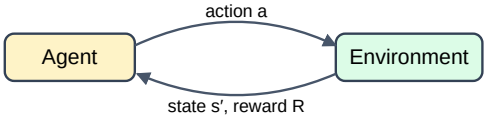
A → B (reward −1); B → terminal 50% (+10), back to A 50% (−1), γ = 1: U(A) = −1 + U(B), U(B) = 5 + 0.5(−1 + U(A)) → U(B) = 8, U(A) = 7.

MDP = fully observable, stochastic, Markovian transition P(s'|s,a), additive rewards; states (start, terminals), actions, transition model, reward R(s,a,s'). Solution = a **policy** (action for every state), not a fixed sequence. Markov: next state depends only on the current one. "MDP defines the task; RL learns how to act in it."

18 Reinforcement learning: passive (direct, ADP, TD) · active (Q-learning)

Workshop 6 ↑ index

RL = learn from trying things when you don't know the transition model. Passive = follow a fixed policy and learn how good states are. Active = also choose actions (explore vs exploit).



RL loop: act, observe the new state and reward, update.

TD: $U(s) \leftarrow U(s) + \alpha\,[R + \gamma U(s') - U(s)]$ (target R + γU(s'); bracket = TD error)

U(s) = 0.02, R = 1, U(s') = 0.57, γ 0.8, α 0.1: target 1.456 → error 1.436 → new U = 0.1636. U = 0.5, U′ = 0.8, R = −0.04, γ 1, α 0.1 → target 0.76, error 0.26 → 0.526.

ADP: $\hat{P}(s'|s,a) = \dfrac{N(s,a,s')}{N(s,a)}$ then solve the (linear) Bellman equations for the fixed policy

(4,3) 2, (3,2) 2, (2,3) 0 → 2/4 = 0.5 · 30 of 40 → 0.75.

Direct utility estimation: reward-to-go = add every reward from s to the end of the trial; average over trials. γ = 1, −0.04 per step, +1 at the end: 4 steps before the goal → 1 − 0.16 = 0.84; −0.04 −0.04 −1 = −1.08. Learns nothing until the trial ends; ignores links between states; slow.

Q-learning: $Q(s,a) \leftarrow Q(s,a) + \alpha\,[R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

Q 2, R 1, γ 0.9, max Q′ 5, α 0.5 → target 5.5 → 2 + 0.5(3.5) = **3.75**. Q 0, R −1, γ 1, Q(s′,·) = {2, −3}, α 0.2 → use max 2 → 0.2. Q 1, R 0, γ 0.9, max 4, α 0.25 → 1.65. Q = .4, R = −.04, γ = .9, max Q′ = .7, α = .5 → .495. Tiny table A→B (0) → G (10), α .5, γ .9: ep1 Q(B) = 5; ep2 Q(A) = 2.25, Q(B) = 7.5.

Method	Model?	Learns	Note
Direct utility	no	U (passive)	average reward-to-go
ADP	**yes** (counts)	U (passive)	adjusts to all successors weighted by P
TD	no	U (passive)	one observed successor; same limit as ADP
Q-learning	no	Q (active)	model-free for learning and acting
DQN	no	Q by a neural net	experience replay + target network

Exploration = try new actions to learn their effects; **exploitation** = use known good ones. Exploration function f(u, n) = R⁺ (optimistic best reward) if n < N_e else u; ε-greedy = random action with probability ε. Sparse rewards make learning hard. Q-learning (off-policy, uses max) vs SARSA (on-policy, uses the action actually taken). Passive = fixed policy, learn U; active = decide what to do.

✗ Answer the new U, not the step. ✗ Multiply U(s') by γ. ✗ Use max over next actions in Q-learning. ✗ "Q-learning needs P(s'|s,a)" = FALSE.

19 Naive Bayes with add-one smoothing

Lecture 8 ↑ index

Score each class = prior × product of the word probabilities in that class; biggest score wins. "Naive" = pretend words are independent given the class.

Spot it: word counts in + and − reviews, T_+, T_-, $|V|$, "classify a review containing …".

Prior P(c) = docs in c / all docs · $P(w|c) = \frac{\text{count}(w,c)+1}{T_c+|V|} \cdot S_c = P(c)\prod_i P(w_i|c) \cdot P(+|\text{text}) = \frac{S_+}{S_+ + S_-}$

Lecture: 11+/9− → priors .55/.45; T_+ 20, T_- 18, |V| 6. heartwarming 6/2 → 7/26 = .27, 3/24 = .13 · delightful .23/.13 · slow .12/.29 · repetitive .08/.29 · charming .27/.08 · dull .04/.08. "heartwarming, delightful, slow, repetitive": S_+ = .000328, S_- = .000640 → P(+) = 0.34 → **negative**. Poll 3 ("…but slow"): S_+ = .00410 > S_- = .00221 → positive. "dull … heartwarming, delightful, charming": 3.69e−4 vs 4.87e−5 → 0.88.
5+/6− reviews, T_+ 26, T_- 24, |V| 6: bottoms 32 and 30; S_+ = 5/11 × (7·1·4·6)/32^4 = 7.28e−5, S_- = 6/11 × (8·1·4·4)/30^4 = 8.62e−5 → P(+) = 0.458 → negative.

✗ Bottom is T + |V|, not + 1. ✗ Dividing by P(text) is not needed to choose the class. ✗ Use only the words in the review. ✗ Unsmoothed zero count wipes out the whole product (that's why we smooth).

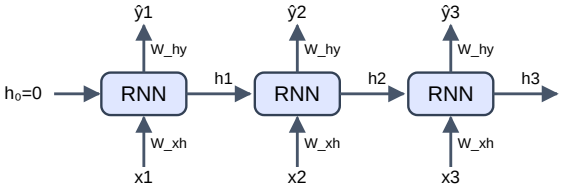
20 RNN hidden-state update · gradients · LSTM

Lecture 8 · key points ↑ index

An RNN reads a sequence one step at a time and keeps a running memory h. New memory = squash(input × its weight + old memory × its weight).

$h_t = \tanh(w_x\, x_t + w_h\, h_{t-1} + b)$ (b only if given) · $\hat{y}_t = W_{hy} h_t$

0.5×2 + 0.25×0.4 = 1.1 → h = tanh(1.1) = **0.800** · 0.2×3 + 0.4×0.5 = 0.8 → 0.664 · 0.6×1 + 0.4×(−0.5) + 0.1 = 0.5 → 0.462 · 0.8 − 0.2 + 0.1 = 0.7 → 0.604 · two steps x = 1, 0.5, w = 0.5, h_0 = 0 → 0.462 → 0.447.



Unrolled RNN: the SAME W_xh, W_hh, W_hy at every step; h_t = tanh(W_hh h_(t−1) + W_xh x_t).

x = 1, 0, 1 with W_xz = .5, W_zz = .8, h_0 = 0: .462, .354, .654. W_xh .5, W_hh .8, W_hy 2, x = 1, .5: h_1 = .462, $\hat{y}_1$ = .924, h_2 = tanh(.25 + .370) = .551, $\hat{y}_2$ = 1.102. 1.5^{10} = 57.7 (explodes), $.5^{10}$ = .001 (vanishes).

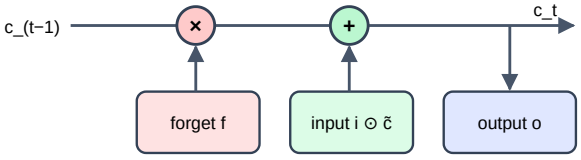
✗ The sum (1.1) is NOT the answer. ✗ Don't swap weights between x and h_prev. ✗ Use the supplied tanh value; round at the end.

Gradient after n steps ≈ factorⁿ: $0.8^{30} \approx 0.001$ (**vanishing** → LSTM/GRU) · $1.2^{30} \approx 237$ (**exploding** → gradient clipping)

Same weights at every step (unrolled = deep net with shared weights). Training = **BPTT**: initialise h_0 → process the sequence and add up losses → backpropagate through time → update shared parameters. Cause of vanishing: repeated × W_hh and tanh′ ≤ 1 → biased to short-term memory. Fixes: activation choice, weight initialisation, gated cells. Design criteria: preserve order, carry context, share parameters, handle variable length. Limits: encoding bottleneck, slow sequential computation, weak long-range memory.

LSTM: $c_t = c_{t-1} \odot f_t + i_t \odot \tanh(\cdot)$ · $h_t = \tanh(c_t) \odot o_t$ · gate = candidate ⊙ gate

c = 0.8, f .9, i .5, o .7, candidate .6 → c = 1.02, h = tanh(1.02)×0.7 = 0.539 · gate [0.8, 0.5, −0.6] ⊙ [1, 0.1, 0.7] = [0.8, 0.05, −0.42].



LSTM: cell updated by ADDING (c_t = c_(t−1)⊙f + i⊙c̃), so gradients survive; h_t = o ⊙ tanh(c_t).

c = 2, f = .9, i = .5, c̃ = 1, o = .8 → c_t = 2.3, h_t = .8·tanh 2.3 = .784.

Forget gate: keep or reset each cell element · **input**: add new information · **output**: expose the cell to h. Cell updated additively → near-constant error flow → long memories. Gates are sigmoids (0–1); candidate is tanh.

21 Text representations · n-grams · sequence structures

Lecture 8 ↑ index

One-hot: length |V|, a single 1 at the word's index (count from 0) · one-hot × E = that word's row of E

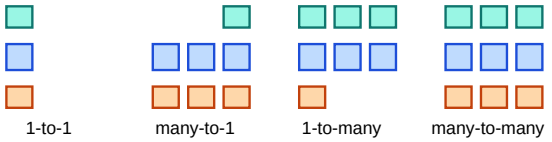
{he, is, singing, she, dancing, stage}: she = index 3 → [0,0,0,1,0,0]. Leia ID 5 → row 5 of E = [−0.95, 0.74, 0.7]. Tokens of "Luke and Leia board the ship with a droid … a Jedi warns them about the Sith": 16 tokens, 14 unique. IDs are names, not sizes.

n-gram: $P(w|h) = \frac{C(h,w)}{C(h)}$ · n-gram = Markov chain of order n − 1

"Mando protects …": Grogu 3, villagers 1, civilians 1 → P(Grogu | Mando protects) = 3/5. Din 5 of 16 → 0.313. "I like AI / I like cats / you like AI": P(AI|like) = 2/3, P(like|I) = 1. Add-one: (0 + 1)/(1000 + 9000) = 1/10000.

One-hot	sparse, all words equally distant, no similarity
Embedding	dense learned vector; similar words close (cosine similarity)
Bag-of-words	word counts, order ignored ("My grandson loved it! So much fun!" → 7 words × 1)
CBOW	context words predict the centre word ("Mando _ Grogu" → protects); average context vectors → softmax → −log p
Skip-gram	centre word predicts each context word
Unknown word	<UNK> token

Firth: "You shall know a word by the company it keeps." cos(a, b) = a·b/(‖a‖‖b‖): (1,2,2)·(2,1,2) = 8/9 = 0.889. v(Paris) − v(France) + v(Italy) ≈ v(Rome); king − man + woman ≈ queen; woman − man + uncle ≈ aunt. n-gram limits: fixed context, no semantic similarity, memorise local patterns. Smoothing: add-one, back-off, interpolation $\lambda_3 P_{tri} + \lambda_2 P_{bi} + \lambda_1 P_{uni}$ (λs sum to 1).



orange = input, blue = RNN cell, teal = output. Many-to-one: sentiment, ECG→diagnosis. One-to-many: captioning, music. Many-to-many: translation, speech→text.

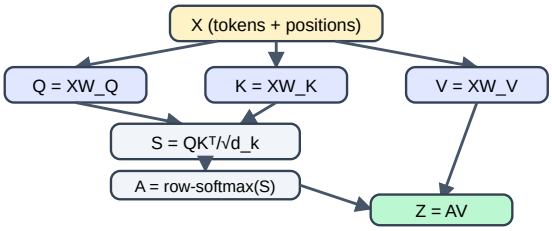
Structure	Examples
many-to-one	ECG → diagnosis · sensor history → failure · video frames → action label · review → sentiment
one-to-many	initial pose → motion · style/seed → music · image → caption
many-to-many	speech → text · translation

22 Scaled dot-product attention

Lecture 8 · key points ↑ index

Attention = a soft search. The query asks; each key says what a token is about; the weights say how much each token matters; the output mixes the VALUES with those weights.

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$



Self-attention: score → scale → softmax (rows sum to 1) → weighted sum of values.

1. Scores $s_i = q \cdot k_i$ (multiply matching parts, add).
2. Scale: $s_i \div \sqrt{d_k}$ ($d_k = 2 \rightarrow \div 1.414$).
3. Softmax: e^s each → add → divide each by the total.
4. Output = $\Sigma\, w_i \cdot v_i$, component by component.

q = [0,1]; k = [1,0], [−2,−1], [2,0]; v = [2,−1], [4,0], [3,4]; d_k = 2 → scores 0, −1, 0 → scaled 0, −0.707, 0 → e: 1, 0.493, 1 (sum 2.493) → w = 0.401, 0.198, 0.401 → output [0.802+0.791+1.203, −0.401+0+1.604] = **[2.797, 1.203]**.
q = [1,0]; k = [1,0],[0,1],[1,1]; d_k 2 → [.71, 0, .71] → w [.40, .20, .40]; values [1,2],[3,0],[0,4] → [0.99, 2.41].

★ Weights only (no q, k)? Skip to step 4: 0.3·[2,0] + 0.7·[0,2] = **[0.6, 1.4]**; 0.2/0.5/0.3 on [1,0,2],[0,1,1],[2,2,0] → [0.8, 1.1, 0.9]. Missing weight = 1 − others (0.2, 0.5 → 0.3).

✗ [0, 2] = picked the biggest weight only · [2, 2] = added without weights. Output length = value length.

Q = XW_Q (what the token asks), K = XW_K (what each token offers for matching), V = XW_V (what gets mixed in). Poll 4 "What was Mando searching for?": Q = question, K = log-token keys, V = separate log-token values. Each row of A sums to 1. Self-attention ignores order → add **positional encoding** PE(pos, 2i) = sin(pos/10000^(2i/d)), PE(pos, 2i+1) = cos(…); position 0 → (0, 1, 0, 1, …). Shapes: X 8×4 · W 4×3 → Q, K, V 8×3 · $QK^\top$ 8×8. Multi-head = several heads with their own W's, concatenated. Transformer block = multi-head attention + feed-forward + residual + layer norm; GPT uses a causal mask. Attention fixes the RNN bottleneck, runs in parallel, reaches distant words directly.

23 Softmax weight · √d_k

Lecture 8 ↑ index

$$w_i = \frac{e^{s_i}}{\sum_j e^{s_j}} \cdot \text{scaled} = \text{raw} \div \sqrt{d_k}$$

Scaled scores [−0.2, 1.7, 1.6, 1.1] → e: 0.819, 5.474, 4.953, 3.004 (sum 14.250) → weight #1 = 0.057. softmax([2,1,0]) = [0.665, 0.245, 0.090]. Raw 6, d_k = 4 → 6/2 = 3 (1.5 = divided by d_k, wrong).

✗ A weight can't be negative or > 1. ✗ Never divide raw scores by their sum. ✓ Equal scores → equal weights. Why √d_k: big dot products make softmax too peaky and gradients vanish.

24 Logic: truth tables · entailment · CNF · resolution

Lecture 10 ↑ index

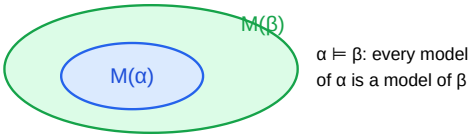
A knowledge base (KB) = facts and rules written in logic. Inference = what MUST be true given them. "KB ⊨ γ" (KB entails γ) means: in every situation where the KB is true, γ is true too.

P	Q	¬P	P∧Q	P∨Q	P⇒Q	P⇔Q
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

★ P ⇒ Q is false ONLY for T ⇒ F (a promise broken). False premise → true ("If 2+2 = 5 then x > 3" is TRUE). Implication ≠ causation.
Precedence (tightest first): **¬, ∧, ∨, ⇒, ⇔** → ¬P ∨ Q ∧ R ⇒ S = ((¬P) ∨ (Q ∧ R)) ⇒ S.

Equivalence	Rule
implication elimination	α ⇒ β ≡ ¬α ∨ β
contraposition	α ⇒ β ≡ ¬β ⇒ ¬α (the converse β ⇒ α is NOT equivalent)
biconditional elimination	α ⇔ β ≡ (α ⇒ β) ∧ (β ⇒ α)
De Morgan	¬(α ∧ β) ≡ ¬α ∨ ¬β · ¬(α ∨ β) ≡ ¬α ∧ ¬β
double negation	¬¬α ≡ α
distributivity	α ∨ (β ∧ γ) ≡ (α ∨ β) ∧ (α ∨ γ)

P = T, Q = F, R = T: P∧Q F · P∨Q T · P⇒Q F · Q⇒R T · P⇔R T · ¬P ∨ Q∧R ⇒ Q: left = F∨F = F → T. P = F, Q = T: P⇒Q T, Q⇒P F, P⇔Q F.



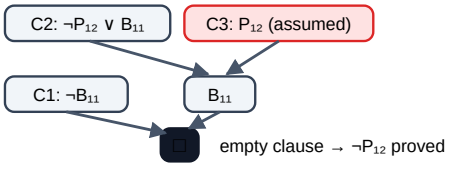
Entailment: M(α) ⊆ M(β).

Three ways to show KB ⊨ γ:

1. **Model checking:** list all 2^n rows; keep rows where KB is true; γ must be true in all of them. KB = P ∧ (P ⇒ Q): only row P=T, Q=T → Q true → entails. KB = P ⇒ Q does NOT entail Q (row F, F). 5 symbols → 32 models.
2. **Theorem proving** with rules: Modus Ponens (α ⇒ β, α ⊢ β), And-elimination (α ∧ β ⊢ α), plus the equivalences.
3. **Proof by contradiction with resolution:** add ¬γ (negate ONLY the query) → convert everything to CNF → resolve pairs with complementary literals → empty clause □ ⇒ KB ⊨ γ (α ⊨ β iff α ∧ ¬β is unsatisfiable).

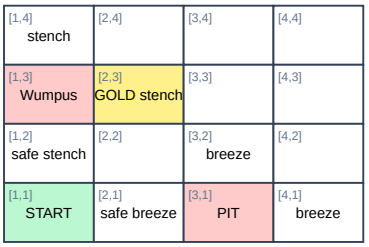
Resolution: $(\ell \vee A),\ (\neg\ell \vee B) \vdash A \vee B$ · CNF steps: ① remove ⇔ ② remove ⇒ ③ push ¬ inward ④ distribute ∨ over ∧

C1 P, C2 ¬P ∨ Q, C3 ¬Q (negated query) → C1+C2 = Q → Q + C3 = □. (P∨Q∨R)+(¬Q∨S) = P∨R∨S · (A∨B)+(¬A∨B) = B · (¬A∨B)+(A∨C) = B∨C.
CNF: A ⇒ (B∧C) = (¬A∨B)∧(¬A∨C) · ¬(A∨B) ⇒ C = A∨B∨C · $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ = $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$.



Resolution refutation: add the negated query, resolve to the empty clause □.

★ Wumpus: KB = ¬B_{11} ∧ ($B_{11} \Leftrightarrow (P_{12} \vee P_{21})$); 3 symbols, 8 models; KB true only at B_{11} = F, P_{12} = F, P_{21} = F ⇒ KB ⊨ ¬P_{12}. Resolution: C1 ¬B_{11}, C2 ¬P_{12} ∨ B_{11}, C3 P_{12} (negated query) → C2+C3 = B_{11} → + C1 = □. Trap: cancel ONE pair at a time: (P∨Q),(¬P∨¬Q) → Q ∨ ¬Q, not □. P ∧ (P ⇒ Q) ∧ ¬Q is not CNF until P ⇒ Q becomes ¬P ∨ Q.



Wumpus world: breeze next to a pit, stench next to the Wumpus. $B_{11} \Leftrightarrow (P_{12} \vee P_{21})$.

Literal = atom or its negation · clause = OR of literals · CNF = AND of clauses · complementary = P and ¬P · satisfiable = true in some

model; unsatisfiable = none (P ∧ ¬P); valid = all. Proposition = declarative statement, true or false (not questions or commands).

24B First-order logic: translate · quantifiers · UI/EI

Lecture 10 ↑ index

FOL talks about OBJECTS (John, x), PROPERTIES/RELATIONS (Student(x), Loves(x,y)) and "for all / there exists", so one sentence covers every student.

FOL: **∀ goes with ⇒, ∃ goes with ∧** · $\neg\forall x\, P \equiv \exists x \neg P$ · $\neg\exists x\, P \equiv \forall x \neg P$

English	FOL
All students study	∀x Student(x) ⇒ Studies(x)
Some student is enrolled in AI	∃x Student(x) ∧ Enrolled(x, AI)
Nothing is perfect	¬∃x Perfect(x) ≡ ∀x ¬Perfect(x)
No student failed	¬∃x Student(x) ∧ Failed(x)
Every parent is older than their child	∀x∀y Parent(x,y) ⇒ Older(x,y)
Everyone loves someone / someone is loved by all	∀x∃y Loves(x,y) / ∃y∀x Loves(x,y) (order matters)

✗ ∀x Student(x) ∧ Studies(x) = "everything is a studying student". ✗ ∃x Student(x) ⇒ Enrolled(x) is true whenever anything is not a student.

Term = names an object (constant John, variable x, function Father(John)); predicate = true/false (Brother(Richard, John), arity 2; zero-ary = propositional symbol like Raining). **Universal instantiation**: replace x by ANY ground term, many times (Student(Mia) ⇒ Studies(Mia)). **Existential instantiation**: replace by a BRAND-NEW constant, once. Then propositionalise → problem: too many ground instances. King(John) ∧ Greedy(John) + ∀x King ∧ Greedy ⇒ Evil → Evil(John). **Deductive** = general rule → certain conclusion; **inductive** = pattern → probable ("Leo will probably pass"). Logical inference applies rules; ML inference applies a trained model.

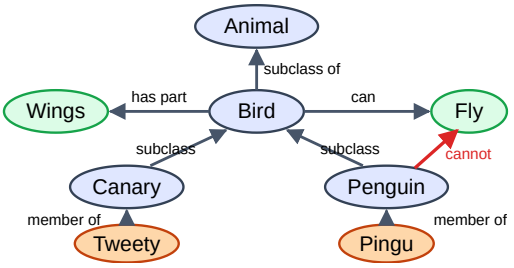
25 Knowledge representation: hierarchies · semantic networks · ontologies · SPARQL

Lecture 12 ↑ index

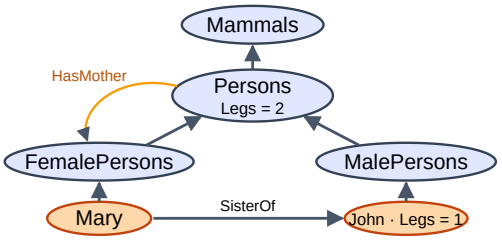
Organise knowledge from general to specific so properties flow down: know it's a cat → it's a mammal → it breathes, without being told.

7 requirements	Meaning
Inclusiveness	top class includes all subclasses
Species / differentia	only one relation between levels: is-a
Inheritance	subclasses take on superclass attributes
Transitivity	A is-a B, B is-a C ⇒ A is-a C
Systematic, predictable rules	grouping/distinguishing rules fixed beforehand
Mutual exclusivity	an entity belongs to only one sibling class
Necessary & sufficient criteria	what must be true / what is enough to belong

Works best: mature, well-understood domain, clear class boundaries, inference from incomplete evidence. **Membership** $BB_9 \in$ Basketballs (object → category); **subset** Basketballs ⊂ Balls (category → category).



Look UP from the object; the first value found wins (Penguin's 'cannot' beats Bird's 'can').



Mary: 2 legs (inherited); John: 1 (own value overrides); Mary's mother ∈ FemalePersons.

Inheritance: start at the object → follow MemberOf / SubsetOf upward → the **first value found wins** (exceptions override defaults)

Tweety ∈ Canary ⊂ Bird (can fly) → flies; Pingu ∈ Penguin (cannot fly) → doesn't fly, but still has wings (from Bird). Mary ∈ Female Persons ⊂ Persons (Legs 2) ⊂ Mammals → 2 legs, a Mammal; John has explicit Legs = 1 → 1. Susan = Mary's mother → HasMother range → Female Persons. Sphynx (a Cat with no fur): Kiki ∈ Sphynx → no fur; Tom ∈ Cat → fur.

Semantic network = graph, nodes = concepts/objects, labelled links; informal, intuitive. **Ontology** = formal, explicit: classes & hierarchy, properties, individuals, axioms → a reasoner can infer reliably: class membership (Student ⊑ Person, Student(Alice) ⇒ Person(Alice)), property chains (enrolsIn ∧ includes ∧ ownedBy ⇒ studiesInDepartment → Alice studies in School of CS), domain/range checks (CB11.07 is a Room → can't enrol), consistency (scheduledIn needs a Room, not a Department). OWL is open-world (missing ≠ false).



Property chain: enrolsIn ∧ includes ∧ ownedBy ⇒ studiesInDepartment.

Knowledge graph = entities + labelled relations; triple **subject – predicate – object**; Wikidata Douglas Adams = Q42.

```
SELECT ?item ?itemLabel ?pic WHERE { ?item wdt:P31 wd:Q146 . ?item wdt:P18 ?pic . SERVICE wikibase:label {…} } → house cats that HAVE an image (P31 = instance of, Q146 = house cat, P18 = image); cats without P18 are left out. Variables start with ?. LIMIT 100. Query types SELECT, CONSTRUCT, DESCRIBE, ASK. RDFS = classes/properties; OWL = more expressive; URI identifies a resource.
```

26 Traps bank: true / false favourites

all modules ↑ index

Statement	
UCS is complete and optimal with positive step costs	T
Greedy best-first is optimal with an admissible h	F
Every admissible heuristic is consistent	F
Graph-search A* is optimal with a consistent h	T
BFS always returns the cheapest path	F
DFS uses less memory than BFS	T
Tree search keeps an explored set	F
Local search keeps the path to the solution	F
SA guarantees the global optimum in one run	F
Tournament selection needs normalised probabilities	F
Fitness 0 can never be picked by roulette	T
A pure branch has entropy 1	F
Information gain can be negative	F
Hyperparameters should be tuned on the test set	F
Accuracy is best for imbalanced data	F
AUC depends on the threshold	F
R^2 can be negative	T
RMSE penalises large errors more than MAE	T
kNN builds an explicit model	F
Features should be scaled before kNN	T
Random forest mainly reduces variance	T
A single perceptron can learn XOR	F
If the perceptron is correct, weights don't change	T
w_zz > 1 in an RNN may explode gradients	T
Value-iteration Bellman equations are linear	F
The solution of an MDP is a fixed action sequence	F
MEU picks the best possible outcome	F
Q-learning needs the transition model	F
TD converges to the same values as ADP	T
A conv layer applies the same weights everywhere	T
Bigger image → more conv parameters	F
Max pooling has trainable parameters	F
Apply softmax before PyTorch CrossEntropyLoss	F
NMS compares boxes of different classes	F
IoU = threshold with a ">" rule → suppressed	F
Detection outputs a fixed number of boxes	F
The pre-activation sum is the RNN hidden state	F
Bag-of-words keeps word order	F
Must divide by P(text) to pick the Naive Bayes class	F
Each row of the attention matrix sums to 1	T
Autoencoder training needs labels	F
After training both G and D generate images	F
Causal masking: parallel training, sequential generation	T
P ⇒ Q means P causes Q	F
"If 2+2 = 5 then x > 3" is true	T
P ∧ ¬P is satisfiable	F
UI used many times; EI once with a new constant	T
A Bayesian network may contain a cycle	F
J is conditionally independent of B given A	T
Inference by enumeration is approximate	F
Semantic-network inheritance stops at the first value found	T
A cat without an image appears with an empty ?pic	F

27 Confusables: pairs the exam mixes up

Book 1 glossary ↑ index

Tree vs graph search	graph search keeps an explored set
Complete vs optimal	finds a solution vs finds the cheapest
Admissible vs consistent	never overestimates vs triangle rule on every edge
Classic vs local search	path matters vs only final state
Parameter vs hyperparameter	learned vs set before training
Bagging vs boosting	parallel, variance vs sequential, bias
Precision vs recall	purity of + predictions vs coverage of actual +
Passive vs active RL	fixed policy, learn U vs choose actions, explore
ADP vs TD	learn model then solve vs update from observed successor
Model-based vs model-free	learns P(s′|s,a) vs learns Q/policy directly
Value vs policy iteration	Bellman with max vs evaluate fixed π then improve
max vs argmax	value (U) vs which action (π)
Classification / detection / segmentation	one label / boxes + labels / label per pixel
Stride vs padding	filter step vs zeros around the border
Max vs average pooling	biggest vs mean (both 0 params)
CBOW vs skip-gram	context → centre vs centre → context
One-hot vs embedding	sparse, no similarity vs dense, similar words close
Query / key / value	need / description to match / content returned
AE vs VAE	fixed code vs distribution (μ, σ) + KL
Causal vs diagnostic	P(effect|cause) vs P(cause|effect)
Deductive vs inductive	certain vs probable
Entailment vs implication	across all models (⊨) vs a connective inside a sentence (⇒)
Semantic network vs ontology	graphical, informal vs formal axioms
Membership vs subset	object ∈ category vs category ⊂ category
Prior vs posterior	before vs after evidence

28 Before you submit

↑ index

1. **Answered what was asked?** order vs cost · new value vs step · % vs decimal · node name vs f-value · value (max) vs action (argmax).
2. **Defaults filled?** conv S = 1, P = 0 · pool S = K · bias only if stated · ties alphabetical.
3. **Kept 4 dp until the end**, then rounded as asked?
4. **Sanity:** probabilities in [0, 1] · weights sum to 1 · IG ≥ 0 · IoU ≤ 1 · entropy ≤ $\log_2$(#classes) · F1 between P and R · ŷ of a sigmoid in (0, 1).
5. **Read the small words:** > vs ≥ · ¬ (use 1 − p) · "select ALL" · "NOT true" · same class only (NMS) · both directions (consistency) · $\log_2$ vs ln.
6. **Multiple answer:** judged each option on its own? **Matching:** every left item used once?